Online Course Retrieval System (OCRS)

Shiven Gupta
Indraprastha Institute of Information Technology
New Delhi, India
shiven19333@iiitd.ac.in

Rishi Raj
Indraprastha Institute of Information Technology
New Delhi, India
rishi19090@iiitd.ac.in

Sushant Gupta
Indraprastha Institute of Information Technology
New Delhi, India
sushant19450@iiitd.ac.in

Samad Shahid
Indraprastha Institute of Information Technology
New Delhi, India
samad19446@iiitd.ac.in

Shashwat Goyal
Indraprastha Institute of Information Technology
New Delhi, India
shashwat19447@iiitd.ac.in

Figure 1: Some online resources

ABSTRACT

There are a vast number of online resources (including courses) available to students and professionals. Their abundance and demand have grown even more during COVID 19 pandemic times. Many educational institutions rely on online resources to supplement their curriculum. However, the resources are scattered around different websites. Therefore, users more often than not spend significant time in "finding the suitable resources". This article presents a proposal for a retrieval system that can ease the process of finding the relevant resources.

KEYWORDS

MOOC, Online Courses, E-Learning, Educational Resources, Information Retrieval, Indexing, Ranking, Spelling Correction, Data Crawling

1 INTRODUCTION

The enormous growth of the internet and increased accessibility to people has made it easier for everyone to learn new things. People can find free and paid courses from the comfort of their homes. This also means that the number of such online courses has increased dramatically. It is difficult to find a good course that is suited for a person's needs. It is not easy to compare courses of different websites or universities. The search for a good course takes countless hours on google. This may lead to people getting discouraged from taking online courses. Thus, the proposal is to build an online course search system, which can gather data from all major online course websites such as Coursera, Edx, Udemy, etc. and present them in an easy to use platform.

2 PROBLEM STATEMENT

Following Problem will be addressed in this project:

- Users often only find a list of resources matching their queries. They are not ranked in order of their relevance. This problem can be handled if a ranking module is introduced in the systems.
- Some of the desired resources can be based on whether the course is free or paid, the difficulty of courses, the unique characteristics of the resource that will match the query.
- Depending on these filters, the system should be able to find out all the resources that will fit the criterion, and then rank them according to the ranking algorithm.
- Project also involves evaluation of the results of the ranked retrieval system.

Figure 2. on following page gives a high level overview of the retrieval system to be worked on.

2.1 Dataset

The dataset is collected and combined from [3, 6–8] to get a unified dataset for Udemy, Coursera, Udacity, EdX. Names and Description from various courses of these websites are combined to form the corpus and the tokens are extracted after following preprocessing:

(1) Punctuation and Special Character Removal
(2) Tokenisation
(3) Stopword Removal

The characteristics of dataset are as follows:

- 45567 Rows : This means 45567 Courses from Websites like Udacity, Udemy, Coursera etc.

- Key Columns : Website Name, Course Name, About Course, Course Link, Difficult Level, Course Rating, Course Categories
- Course Categories can further be divided into : Tech (9964 Courses), Marketing (8788 Courses, Finance (5887 Courses), Design (9962), Business (9970)
- Difficult Level can be of Type : Beginner (21531 Courses), Intermediate (15238 Courses), Advanced (12307 Courses)
- Ratings are in the range (1,5) with max 1 decimal allowed
- The Vocabulary made out of dataset contains 57513 terms. Thus the document term matrix used in the TF-IDF model in subsequent sections has the dimension (45567,57513).

3 LITERATURE REVIEW

Many literature studies have assessed the need and scope of a retrieval system that can use exisiting MOOC websites to come up with combined results.

One such study [2] explores retrieving and mining techniques relevant to this work. The authors review CLIR (cross language Information Retrieval), Vertical and Tropical Search Engines and their implementation in a Unified MOOCs Search Engine. Most of the existing work comprise of three types of systems vis a vis Only Search, Only Recommender System, Integration of both Search and Recommendation. The study [2] mentions existing works like Courducate [4], MoocRec.com [10] etc, alongwith their shortcomings. Then authors [2] propose a system MRSE. It has web crawling as its core data collection component which uses technologies like Solr Nutch (crawling module). Then it implements indexing module which indexes the data, then clusters it and then implements ranking algorithms. Then they work on querying module which improves retrieval of courses in Arabic and English Language.

For ranking purposes, students from University of Illinois, Urbana-Champaign have used the MB-25 and Dirichlet prior algorithms and the BM25 algorithm in their implementation of Courducate [4]. Also, they suggest 'Spell Check' and 'Recommendations' as future work in their project. Their project can even be expanded to include more number of courses and related resources.

For more specific course retrieval, this thesis [11] uses semantic similarity techniques to identify the MOOCs offered by the e-learning and compare them to the courses offered in a university. The student can compare his or her university's normal courses with those given by e-learning websites, and enrol in similar online courses to gain a better knowledge of the topic. The paper also explores ways to index, store and extract data from different online sources while giving sample training data.

In the paper titled "Hybrid massive open online course (MOOC) recommendation system using machine learning" [5], the authors take a hybrid approach to the recommendation system. Instead of using the complete dataset, they use a small subset of similar users to produce predictions. The system first generates implicit ratings from user behavior. This is done by taking the ratio of time spent by user and total duration of the course. The ratings are represented in the form of a 2d matrix showing the ratings of each user for each course. This is then used to cluster similar users by using the k-means clustering algorithm. They then use collaborative filtering to train the system. The paper by Pang et al. [9] explains collaborative filtering for MOOC in great depth. After training the parameter vectors and feature vectors are generated, ratings for an unregistered course by a user is predicted and courses are recommended by top ratings.

In another paper on recommendations [12], the authors have made a recommendation system based on distributed computational framework. They have improved upon the already existing Apriori algorithm which was used earlier. Data on courses is imported into a Hadoop file system through the use of Sqoop, preprocessed in the standard form, and then simplified and stored in HDFS. Analysis is then done using Spark. Spark is able to read from HDFS for calculations. Finally, the system recommends a course depending on the parameters. Ranking is done using confidence score, and the results are exported through Sqoop.

```
Did you mean : learn deep learning python program  ?
https://www.coursera.org/learn/teach-java-arraylist-2d-arrays
https://www.coursera.org/learn/social-media-advertising-fundamentals
https://www.coursera.org/learn/tensorflow-for-ai-applying-image-convolution
https://www.coursera.org/learn/tensorflow-for-ai-neural-network-representation
https://www.udemy.com/course/industry-orientated-solidworks-training-most-essential/
https://www.coursera.org/learn/neural-networks-deep-learning
https://www.coursera.org/learn/deep-neural-network
https://www.coursera.org/learn/teach-java-sequences-primitive-types-object
https://www.coursera.org/learn/chimp
https://www.coursera.org/learn/nand2tetris2
https://www.coursera.org/learn/customising-models-tensorflow2
```

Figure 2: Number of Hits Model on Query "learn deep learming pthon programing"

```
learn deep learning python program
https://www.udemy.com/course/data-science-linear-regression-in-python/
https://www.udemy.com/course/python-for-computer-vision-with-opencv-and-deep-learning/
https://www.udemy.com/course/industry-orientated-solidworks-training-most-essential/
https://www.udemy.com/course/keras-deep-learning-in-python/
https://www.udemy.com/course/deep-learning-by-tensorflow-tfkeras-keras-using-python/
https://www.udemy.com/course/complete-datascience-with-python-and-tensorflow/
https://www.udemy.com/course/pro-data-science-in-python/
https://www.udemy.com/course/learning-path-r-advanced-deep-learning-with-r/
https://www.udemy.com/course/complete-tensorflow-2-and-keras-deep-learning-bootcamp/
https://www.udemy.com/course/learn-data-science-deep-learning-machine-learning-nlp-r/
```

Figure 3: BM25 Model on Query "learn deep learming pthon programing"

4 BASELINE MODEL AND RESULTS

After compiling the dataset and the required preprocessing, two baseline models are implemented to review the capability and shortcomings of the models as required and mentioned in the problem statement.

The first model uses number of hits in the corpus i.e the intersection of the tokens in the query and all the courses and then sorts the courses with the highest number of hits first. In case of common number of hits, the courses are ranked according to their ratings. The results of the model when it is run on "Computer Science Python Code for Factorial of a Number" query is displayed in Figure 3 alongwith the course hits and the course ratings. Figure 4 shows the distribution of Courses according to the number of hits.

Figure 4: Number of Hits Model and with filter for rating > 4 on Query "learn deep learming pthon programing"

Most of courses have 0 hits with only 1 course having maximum of 5 hits and 11 courses having 4 hits.

The second model uses tfidf scores (term frequency - inverse document frequency) to rank the courses. For this document term weight matrix is created and stored, with each cell storing the corresponding tfidf value. In this model, the query is processed into individual terms and the tfidf values for each term against all the courses are retrieved from the matrix. This is done for every term in the query and then tfidf values of all terms are added for each course. The course having highest sum is ranked first. The results of this model are displayed in Figure 4 alongwith their corresponding tfidf scores.

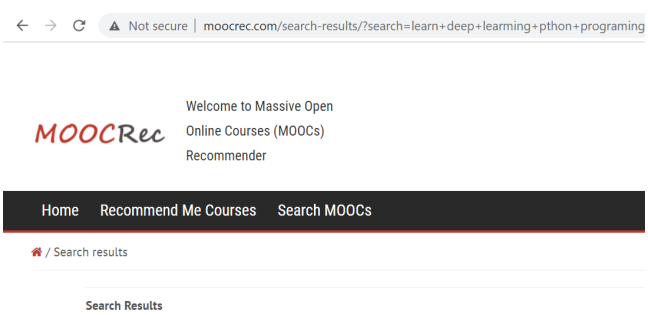

Figure 5: MOOCRec Null Response on Query "learn deep learming pthon programing"

5 FINAL METHODOLOGY

The first model used in the previous section doesnt solve the problem mentioned in the problem statement vis-a-vis giving the user the option of filters for better retrieval. This can be solved by adding filter functions that filter the retrieved courses based on user specifications. The other important addition is spell check mechanism which is absent in [4]. The spell check is implemented by using ngrams and jaccard distance to suggest the user with the correct spelling. As an improvement to second model, the BM25 model is used for ranked retrieval.

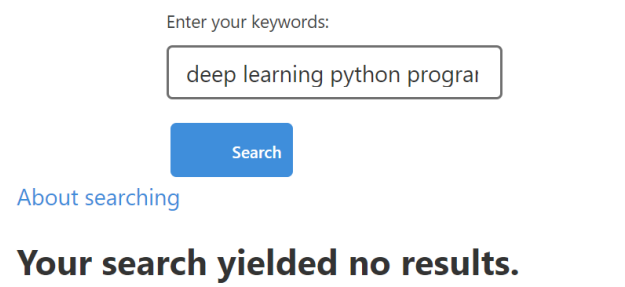

Figure 6: mooc-list Null Response on Query "learn deep learming pthon programing"

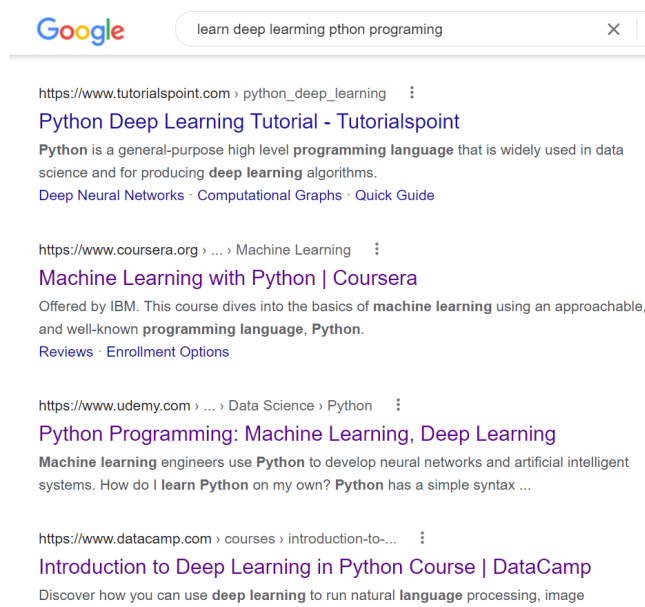

Figure 7: Google Response on Query "learn deep learming pthon programing"

6 EVALUATION

We compare our results with MOOC Retrieval Systems already in place like Moocrec.com, Mooc-List.com, classcentral.com. We also enter query on Google to compare.

7 CONTRIBUTIONS

- The data collected from various course websites [3, 6–8] is compiled and saved in the Github Repository[1].
- The spell check implementation to allow for quicker but correct results thus removing the need for user to enter exact same spelling to retrieve the courses.
- Addition of filters to give users more control over their search results.

8 FUTURE WORK

Our system uses a fixed and already compiled dataset for retrieval which can be improved by developing ways to add new data to already existing data or collect data dynamically after starting the system.

9 CONCLUSION

In this project we develop a retrieval system which outputs relevant course links according to user's input. The corpus is subset of existing courses on MOOC Websites like Udemy, Udacity, Coursera, EdX. The features of the system are spell check, ranked retrieval of courses, and filters based on ratings, difficulty and websites. The evaluation is done against the baseline models and against similar systems mooclist and classcentral and our system gives at par or even better results on most of the queries.

REFERENCES

[1] Team 31. 2022. *MOOC Retrieval System.* Retrieved April,2022 from https://github.com/shashgoyal/IR2022_project_31

[2] Khadijah M. Alzahrani and Maram Meccawy. 2021. MOOCs One-Stop Shop: A Realization of a Unified MOOCs Search Engine. *IEEE Access* 9 (2021), 160175–160185. https://doi.org/10.1109/ACCESS.2021.3130841

[3] Shubham Chambhare. 2020. *All major fields courses in Udemy - (44K+ Courses).* Retrieved March,2022 from https://www.kaggle.com/datasets/shubhamchambhare/all-major-fields-courses-in-udemy-44k-courses/metadata

[4] Qian Cheng and Yunlong Gao. 2015. *Courducate – An MOOC Search and Recommendation System.* Retrieved Feb 24, 2022 from http://sifaka.cs.uiuc.edu/~wang296/Course/IR_Fall/docs/Projects/Samples/2.pdf

[5] Vishal Garg and Ritu Tiwari. 2016. Hybrid massive open online course (MOOC) recommendation system using machine learning. In *International Conference on Recent Trends in Engineering, Science Technology - (ICRTEST 2016).* 1–5. https://doi.org/10.1049/cp.2016.1479

[6] Khushee Kapoor. 2021. *Coursera Courses Dataset 2021.* Retrieved March,2022 from https://www.kaggle.com/datasets/khusheekapoor/coursera-courses-dataset-2021

[7] Khushee Kapoor. 2021. *EdX Courses Dataset 2021.* Retrieved March,2022 from https://www.kaggle.com/datasets/khusheekapoor/edx-courses-dataset-2021

[8] Khushee Kapoor. 2021. *Udacity Courses Dataset 2021.* Retrieved March,2022 from https://www.kaggle.com/datasets/khusheekapoor/udacity-courses-dataset-2021

[9] Yanxia Pang, Yuanyuan Jin, Ying Zhang, and Tao Zhu. 2017. Collaborative filtering recommendation for MOOC application. *Computer Applications in Engineering Education* 25, 1 (2017), 120–128. https://doi.org/10.1002/cae.21785 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/cae.21785

[10] Panagiotis Symeonidis and Dimitrios Malakoudis. 2018. *MoocRec.com: Massive Open Online Courses Recommender System: Algorithms, Practical Challenges and Applications.* 627–651. https://doi.org/10.1142/9789813275355_0019

[11] Krishna Nitin Tenali. 2014. *Semantic Similarity based Information Retrieval as applied to MOOCs.* Master's thesis. San Jose State University. https://doi.org/10.31979/etd.rejz-tj5q

[12] Hao Zhang, Tao Huang, Zhihan Lv, SanYa Liu, and Zhili Zhou. 2018. MCRS: A course recommendation system for MOOCs. *Multimedia Tools and Applications* 77, 6 (March 2018), 7051–7069. https://doi.org/10.1007/s11042-017-4620-2